

PROJECT DOCUMENTATION

Food Delivery Management

A Software Solution for Efficient Online Food Ordering, Restaurant Operations and Delivery Coordination

Team Member	Project Title	Roll Number
Tushar Kumar	Food Delivery Management	2400290120267
Tapish Chaudhary	Food Delivery Management	2400290120262
Vinayak Tyagi	Food Delivery Management	2400290120277
Vivek	Food Delivery Management	2400290120281

Software Project Report

Prepared as a team project for academic evaluation

September 2026

Table of Contents

Section	Topic	Page
1	Project Overview	3
2	Background and Context	4
3	Problem Statement	5
4	Impact Analysis	6
5	Proposed Software Solution	7
6	System Features and Modules	8
7	Functional Requirements	9
8	Non-Functional Requirements	10
9	Technical Feasibility	11
10	Operational Feasibility	12
11	Project Scope – In Scope	13
12	Project Scope – Out of Scope	14
13	System Workflow and Data Design	15
14	Expected Outcomes and Benefits	16
15	Conclusion and Future Enhancements	17

Document purpose

This document defines the concept, problem, proposed solution, feasibility, scope and expected outcomes of the Food Delivery Management project. It is intended to provide a clear academic and technical description of the proposed system and its value to customers, restaurants, delivery personnel and administrators.

Project vision

The vision is to create a simple, reliable and scalable platform through which customers can discover food, place orders, make payments, receive status updates and provide feedback, while restaurants and delivery personnel can manage their respective operations from dedicated interfaces.

1. Project Overview

Food Delivery Management is a software application designed to coordinate the complete lifecycle of an online food order. The platform connects customers with restaurants and delivery personnel through a centralized digital system. Instead of depending on manual phone calls, fragmented messaging or disconnected records, the proposed solution provides one workflow from menu discovery to order completion.

The project focuses on three core user groups: customers, restaurant staff and delivery personnel. An administrator supervises users, restaurants, menus, orders, payments and system-level activities. Each role receives functionality appropriate to its responsibilities.

For customers, the system provides account creation, restaurant browsing, menu viewing, cart management, address selection, order placement, payment selection, order tracking, order history and feedback. For restaurants, it provides menu management, incoming order management and status updates. Delivery personnel can view assigned orders, update pickup and delivery status, and complete deliveries.

The application can be implemented as a web-based or responsive application backed by a relational database and a server-side API. The architecture should keep user interface, business logic and data management separated so that the system can be expanded later.

Key objectives

- Digitize the food ordering and delivery workflow.
- Reduce errors caused by manual order recording and communication.
- Provide customers with clear order status and delivery information.
- Give restaurants a structured way to manage menus and incoming orders.
- Improve coordination between restaurants, customers and delivery personnel.
- Maintain organized records of users, orders, payments and feedback.
- Provide an extensible foundation for future features such as offers, ratings and analytics.

Primary stakeholders

Customers are the end users who place orders. Restaurants prepare and manage orders. Delivery personnel transport orders. Administrators maintain system data and monitor operations. The development team is responsible for design, implementation, testing, deployment and maintenance.

2. Background and Context

Food ordering has increasingly shifted from offline interactions to digital platforms. Customers expect convenient access to menus, transparent pricing, multiple payment choices and information about order progress. Restaurants similarly need an organized mechanism to receive orders and manage preparation without relying entirely on handwritten notes or informal communication.

A food delivery process contains multiple stages: customer discovery, restaurant selection, menu selection, cart creation, checkout, payment, restaurant confirmation, food preparation, pickup, delivery and completion. Each stage creates information that should be recorded consistently. When these stages are handled independently, mistakes can occur in item selection, quantities, pricing, addresses or status communication.

The proposed project treats the process as one integrated workflow. The system stores the customer and restaurant information, maintains menu data, creates an order record, calculates totals and tracks status changes. This improves visibility because every authorized participant can see the part of the process relevant to them.

Academic and practical relevance

From an academic perspective, the project demonstrates the application of software engineering concepts including requirements analysis, database design, authentication, role-based access, CRUD operations, transaction processing, validation, testing and deployment. From a practical perspective, it represents a common real-world information system where several user roles interact with shared data.

Assumptions

- Users have internet-enabled devices for accessing the application.
- Restaurants maintain accurate menu and availability information.
- Delivery personnel can access assigned orders and update statuses.
- Payment processing is represented through an integrated or simulated payment workflow depending on the project environment.
- The initial version targets a manageable geographic service area rather than nationwide operations.

The background establishes why a centralized application is useful: it turns a sequence of disconnected tasks into a traceable and manageable digital process.

3. Problem Statement

The central problem is the lack of a unified and structured mechanism for managing customers, restaurants, food menus, orders, payments and deliveries in one workflow. In a manual or fragmented environment, order information can be duplicated, delayed or recorded incorrectly. Customers may not know whether an order has been accepted, prepared, picked up or delivered.

Specific problems

- Manual order taking can lead to incorrect food items, quantities and customer details.
- Customers may have limited visibility into the progress of an order.
- Restaurants need a simple interface to receive and organize multiple orders.
- Delivery assignments and status updates can become difficult to coordinate without a shared system.
- Order history and customer feedback may not be stored in a consistent format.
- Price calculations can be affected by manual arithmetic or outdated menu information.
- Administrators may lack a single dashboard for monitoring users, restaurants and orders.
- Disconnected communication increases the possibility of delays and misunderstandings.

Problem definition

The project therefore aims to develop a centralized Food Delivery Management system that records the complete order lifecycle and provides role-specific interfaces. The system should reduce avoidable manual work while improving transparency and data consistency.

Research and design questions

- How can a single workflow connect customer ordering, restaurant processing and delivery?
- How can order status be represented clearly and updated securely?
- How can user roles be separated so that each participant accesses only relevant functions?
- How can order and payment records be stored so that the system remains consistent?
- How can the design support future expansion without rebuilding the complete application?

The problem is not simply food ordering; it is the coordination of multiple operational actors around one order. The proposed software addresses this coordination problem through shared, structured data and controlled workflows.

4. Impact Analysis

A well-designed Food Delivery Management system can create positive impact at customer, restaurant, delivery and administrative levels. The most important impact is improved coordination: participants work from the same order record rather than maintaining separate information.

Customer impact

- Faster ordering through searchable restaurants and structured menus.
- Better visibility through order status updates.
- Reduced probability of incorrect orders through cart and checkout confirmation.
- Convenient access to previous orders and feedback options.
- Improved transparency of item totals, delivery charges and final amount.

Restaurant impact

- Centralized incoming order management.
- Faster updates of menu items, prices and availability.
- Clear preparation and order status workflow.
- Reduced dependency on manual order registers.
- Better access to order history for operational review.

Delivery impact

- Structured assignment of delivery tasks.
- Clear pickup and delivery information.
- Simple status updates that keep customers and restaurants informed.
- Reduced confusion caused by repeated communication.

Administrative impact

- Centralized user and restaurant management.
- Visibility into order records and system activity.
- Improved ability to identify common operational issues.
- A foundation for future analytics and reporting.

Broader impact

The system supports digital transformation by converting operational information into structured records. It can also reduce paper usage and repetitive manual data entry. The impact will depend on implementation quality, usability, data accuracy and the operating procedures adopted by participating restaurants.

5. Proposed Software Solution

The proposed solution is a role-based Food Delivery Management application. It consists of a customer interface, restaurant interface, delivery interface and administrator interface connected to a common backend and database.

Core solution workflow

- Customer registers or logs in.
- Customer browses available restaurants and menus.
- Customer adds selected food items to the cart.
- System validates item availability and calculates the order total.
- Customer confirms address and payment method.
- System creates the order and records its initial status.
- Restaurant receives the order and accepts or rejects it.
- Accepted order moves through preparation.
- Delivery personnel receives or accepts the delivery assignment.
- Delivery status changes to picked up, out for delivery and delivered.
- Customer receives the completed order and can submit feedback.

Role-based access

Authentication and authorization should be used to separate permissions. Customers should not be able to modify restaurant menus, delivery personnel should not modify customer accounts, and restaurant users should only manage information belonging to their restaurant. Administrative functions should be protected by elevated privileges.

Validation and reliability

The system should validate required fields, quantities, prices, addresses and status transitions. Server-side validation is essential because client-side checks alone can be bypassed. Database constraints should further protect the integrity of orders and relationships.

Scalability approach

The system should use modular components and well-defined APIs. This makes it possible to later add coupons, subscriptions, live maps, ratings, loyalty programs, multiple restaurants per order, delivery optimization and analytics without redesigning the complete foundation.

6. System Features and Modules

The application is divided into modules so that each business responsibility can be developed and tested independently.

Module	Purpose
User Management	Registration, login, profile management, address management and role-based access.
Restaurant Management	Restaurant profile, operating information, menu categories, items, prices and availability.
Food Catalog	Display of food items with descriptions, prices, categories and availability.
Cart and Checkout	Item quantity changes, subtotal calculation, delivery charge, total calculation and order confirmation.
Order Management	Order creation, confirmation, preparation, assignment, pickup, delivery and completion.
Payment Management	Payment method selection, payment status and transaction reference handling.
Delivery Management	Delivery assignment, pickup details, delivery status and completion.
Feedback and Ratings	Customer feedback and optional rating after order completion.
Admin Management	Management of users, restaurants, orders and selected system-level records.

The modular structure improves maintainability because changes to one area can be isolated. For example, adding a new payment provider should primarily affect the payment service rather than the restaurant menu module.

Suggested dashboards

- Customer dashboard: active orders, order history and saved addresses.
- Restaurant dashboard: incoming orders, active preparation and menu controls.
- Delivery dashboard: assigned deliveries and current delivery statuses.
- Admin dashboard: users, restaurants, orders and system summaries.

7. Functional Requirements

Functional requirements describe what the system must do.

ID	Function	Requirement
FR-01	User registration	The system shall allow a new customer to create an account using validated information.
FR-02	Authentication	The system shall authenticate users and establish an authorized session.
FR-03	Role management	The system shall enforce permissions based on customer, restaurant, delivery or admin role.
FR-04	Restaurant browsing	The system shall display available restaurants and relevant information.
FR-05	Menu management	Authorized restaurant users shall create, update and deactivate menu items.
FR-06	Cart management	Customers shall add, remove and update quantities of food items.
FR-07	Checkout	The system shall calculate subtotal, applicable charges and final order amount.
FR-08	Order creation	The system shall create an order record after successful confirmation.
FR-09	Order status	Authorized users shall update order status according to the defined workflow.
FR-10	Delivery assignment	The system shall associate an active order with a delivery person.
FR-11	Payment status	The system shall record payment method and payment status.
FR-12	Order history	Customers shall view completed and previous orders.
FR-13	Feedback	Customers shall be able to submit feedback after an eligible order.
FR-14	Administration	Administrators shall manage core records and monitor system activity.

8. Non-Functional Requirements

Non-functional requirements define the quality characteristics expected from the application.

Quality Area	Expected Standard
Performance	Common pages and API requests should respond quickly under normal project-scale load.
Security	Passwords should be protected using secure hashing; authentication and authorization must be enforced.
Usability	The interface should be simple enough for first-time users to understand the ordering workflow.
Reliability	Orders should not be lost during normal application operation, and important failures should be logged.
Availability	The deployed application should be accessible during the intended service period.
Scalability	The architecture should allow additional users, restaurants and orders without major redesign.
Maintainability	Code should be modular, documented and organized around clear responsibilities.
Data Integrity	Relationships, required fields and valid status transitions should be protected.
Compatibility	The responsive interface should work on common desktop and mobile browsers.
Privacy	Only required customer information should be collected and access should be restricted by role.

Security considerations

Authentication should use secure password storage and protected sessions or tokens. Input must be validated and sanitized. Authorization should be checked at the API or service layer, not only in the interface. Sensitive configuration such as database credentials and API secrets should be stored outside source code. Administrative actions should be logged where practical.

Data integrity considerations

An order should reference a valid customer and restaurant, and its items should reference valid menu items. Totals should be calculated consistently. Status changes should follow an allowed sequence to avoid impossible states such as marking an order delivered before it is picked up.

9. Technical Feasibility

Technical feasibility evaluates whether the proposed system can be built using commonly available software and development skills. The project is technically feasible because its core functions—authentication, CRUD operations, relational data management, REST-style APIs and responsive interfaces—are established software development patterns.

Suggested technology stack

Layer	Suggested Choice
Frontend	HTML, CSS, JavaScript or a modern component-based framework such as React.
Backend	Node.js with Express, or another REST-capable backend framework.
Database	PostgreSQL or MySQL for structured relational storage.
Authentication	Session-based authentication or JWT with secure password hashing.
Version Control	Git and a shared remote repository for collaboration.
Deployment	Cloud or institutional server environment suitable for the chosen stack.
Testing	Unit tests, API tests and end-to-end workflow testing.

Technical architecture

A three-layer architecture is recommended. The presentation layer contains the customer, restaurant, delivery and admin interfaces. The application layer contains authentication, business rules, order processing, payment handling and delivery logic. The data layer contains the relational database and persistence mechanisms.

Technical risks and mitigation

- Risk: inconsistent order state → Mitigation: define and validate allowed status transitions.
- Risk: unauthorized access → Mitigation: role-based authorization on protected APIs.
- Risk: incorrect totals → Mitigation: server-side calculation and database-backed item prices.
- Risk: data loss → Mitigation: regular database backups and transaction-safe operations.
- Risk: collaboration conflicts → Mitigation: Git branches, code reviews and clear module ownership.

Conclusion

The required technology is accessible, mature and suitable for a student-scale project. The system can be developed incrementally, beginning with authentication, restaurants, menus and orders, followed by delivery, payments and feedback.

10. Operational Feasibility

Operational feasibility examines whether the proposed system can realistically be used by its intended stakeholders. The system is designed around familiar actions—browse, add to cart, checkout, accept order, prepare, pick up and deliver—so the learning curve should be manageable.

Customer operations

Customers interact with a guided ordering process. The interface should clearly distinguish available items, cart contents, total amount and order status. Confirmation screens should reduce accidental submissions.

Restaurant operations

Restaurant staff should receive incoming orders in a dashboard and update the order as preparation progresses. Menu management should be simple enough for non-technical staff to change availability and prices.

Delivery operations

Delivery personnel should see assigned deliveries and essential information such as pickup location, customer address and order identifier. Status controls should be limited to valid actions to prevent incorrect updates.

Administrative operations

Administrators should have a concise dashboard for managing accounts, restaurants and order records. Administrative screens should expose only the information required for operational management.

Training and support

- Provide a short user guide or onboarding instructions.
- Use clear labels, confirmations and error messages.
- Provide basic troubleshooting guidance for login, order and payment issues.
- Maintain a support or administrator contact for operational issues.

Change management

Introducing the system may require restaurants and delivery staff to change existing habits. A pilot deployment with a small number of users can help identify usability problems before wider adoption. Feedback should be collected and converted into prioritized improvements.

Operational conclusion

The project is operationally feasible because the proposed workflows align with common food ordering activities and can be introduced gradually. Success depends on usability, reliable connectivity, accurate data entry and user acceptance.

11. Project Scope – In Scope

The project scope defines the functionality intended for the initial version. Keeping the scope controlled helps the team deliver a stable core system within an academic project timeline.

In-scope functionality

- Customer registration, login and profile management.
- Restaurant registration or administrator-managed restaurant records.
- Restaurant browsing and basic search/filtering.
- Menu categories and food item management.
- Food item availability and price management.
- Cart creation and quantity updates.
- Checkout with address and order summary.
- Order creation and order history.
- Order status tracking from placement to delivery.
- Restaurant acceptance/rejection and preparation status.
- Delivery assignment and delivery status updates.
- Payment method and payment status recording.
- Customer feedback and basic ratings after completed orders.
- Administrator management of users, restaurants and orders.
- Database storage for core entities and relationships.
- Basic validation, error handling and role-based authorization.

Deliverables

- Requirements and project documentation.
- Database schema and application source code.
- Customer-facing ordering interface.
- Restaurant management interface.
- Delivery management interface.
- Administrative interface.
- Testing evidence for major workflows.
- Deployment-ready application build.

Scope control

Features that are not essential to the core ordering lifecycle should be treated as optional enhancements. This ensures that the project prioritizes a working end-to-end workflow over a large number of partially implemented features.

12. Project Scope – Out of Scope

The following items are outside the initial project scope. They may be considered future enhancements after the core application is stable.

- Nationwide or international multi-region logistics.
- Advanced real-time GPS routing and fleet optimization.
- AI-based demand forecasting or personalized recommendations.
- Automated voice ordering through assistants.
- Complex loyalty programs and subscription memberships.
- Large-scale advertising and sponsored restaurant placement.
- Advanced fraud detection and enterprise risk scoring.
- Multi-vendor cart checkout across unrelated restaurants in one transaction.
- Sophisticated warehouse or inventory management.
- Full accounting, payroll and restaurant enterprise resource planning.
- Drone or autonomous delivery integration.
- Large-scale data warehouse and business intelligence platform.
- Advanced dynamic pricing based on demand and traffic.
- International tax, currency and regulatory handling.

Reason for exclusions

These features require additional infrastructure, integrations, algorithms, operational policies or domain-specific expertise. Including them in the first release would increase complexity and testing effort. Excluding them does not mean they are impossible; it means they are intentionally deferred.

Scope boundary

The boundary of the project is a reliable core food delivery workflow: user access, restaurant/menu discovery, cart, checkout, order processing, delivery status and basic feedback. Any feature that does not directly support this lifecycle can be evaluated separately.

Priority principle

If development time becomes limited, the team should prioritize the successful completion of the customer-to-order-to-delivery workflow, database integrity, authentication and testing before adding optional features.

13. System Workflow and Data Design

The system revolves around the Order entity. A customer places an order containing one or more order items. Each order is associated with a restaurant, payment information and, when applicable, a delivery assignment.

High-level workflow

Stage	Actor	System Action
1. Discover	Customer	Browse restaurants and menus.
2. Select	Customer	Add food items and quantities to cart.
3. Checkout	Customer	Confirm address, payment method and total.
4. Create Order	System	Persist order and order items.
5. Accept	Restaurant	Accept or reject the order.
6. Prepare	Restaurant	Prepare food and update status.
7. Assign	Admin/System	Assign delivery personnel.
8. Pickup	Delivery	Collect order and update status.
9. Deliver	Delivery	Deliver order and mark completed.
10. Feedback	Customer	Submit optional rating/feedback.

Core database entities

- User: user_id, name, email/phone, password hash, role and profile data.
- Restaurant: restaurant_id, name, contact details, address and status.
- MenuItem: item_id, restaurant_id, category, name, description, price and availability.
- Order: order_id, customer_id, restaurant_id, total, address, payment status and order status.
- OrderItem: order_item_id, order_id, item_id, quantity and price snapshot.
- Delivery: delivery_id, order_id, delivery_person_id and delivery status.
- Feedback: feedback_id, order_id, customer_id, rating and comment.

Price snapshot principle

The order item should store the applicable price at the time of ordering. This prevents historical orders from changing when a restaurant later changes its menu price.

14. Expected Outcomes and Benefits

The expected outcome is a functional prototype or deployable academic application that demonstrates an end-to-end food delivery process. The system should allow the team to show the complete flow from customer login to order completion.

Expected outcomes

- A centralized database for users, restaurants, menus and orders.
- A working customer ordering workflow.
- A restaurant dashboard for menu and order management.
- A delivery workflow for assigned orders and status changes.
- Basic payment and transaction status handling.
- Order history and customer feedback functionality.
- Role-based access and validation.
- Documented requirements, design decisions and testing approach.

Measurable indicators

- Percentage of core user journeys completed successfully during testing.
- Number of critical defects remaining at final demonstration.
- Correctness of order totals across representative test cases.
- Successful enforcement of role-based permissions.
- Successful completion of order status transitions.
- User feedback on clarity and ease of the ordering workflow.

Benefits

The project provides a practical example of how software can coordinate a multi-actor business process. It can reduce manual record keeping, improve visibility of orders, standardize status handling and provide a foundation for data-driven improvements.

Academic learning outcomes

- Requirement analysis and software project planning.
- Database modeling and relational data management.
- Frontend-backend integration.
- API design and authentication.
- Role-based authorization.
- Testing and debugging.
- Git-based team collaboration and version control.
- Documentation and presentation of a real-world software solution.

15. Conclusion and Future Enhancements

Conclusion

Food Delivery Management is a practical software project that addresses the coordination challenges involved in digital food ordering and delivery. The proposed system creates a shared workflow connecting customers, restaurants, delivery personnel and administrators. Its core value lies in organizing information and responsibilities around a single order lifecycle.

The project is technically feasible using widely adopted web technologies and relational database systems. It is also operationally feasible because the proposed workflows correspond to familiar real-world tasks. A controlled scope ensures that the team can prioritize a stable core application before introducing advanced functionality.

Future enhancements

- Live map-based delivery tracking.
- Estimated delivery time calculation.
- Coupon, discount and promotional campaign management.
- Restaurant and food ratings with moderation.
- Personalized food recommendations.
- Loyalty points and customer rewards.
- Push notifications, SMS or email notifications.
- Advanced restaurant analytics and sales dashboards.
- Automated delivery-person assignment.
- Demand forecasting and intelligent delivery optimization.
- Multiple payment gateway integrations.
- Cloud scaling, caching and observability for larger workloads.

Final project statement

The proposed Food Delivery Management system provides a strong foundation for demonstrating software engineering principles through a real-world application. By focusing first on correctness, usability, security and a complete end-to-end workflow, the team can produce a meaningful and extensible project suitable for academic evaluation and future development.

Team Members

Tushar Kumar — 2400290120267

Tapish Chaudhary — 2400290120262

Vinayak Tyagi — 2400290120277

Vivek — 2400290120281